

If you want to automate, you learn to code

- and you learn to code well

Fast paced summary of what we are going to cover

insert fast paced summary here :)

In this talk we're going to look at how we can learn to automate things. And by things I mean software.

Very often people take that to mean "Test Automation". But we're not going to do that. We are going to talk about automating software and if that forms part of your test process then that is great, so long as you do it well.

And we are going to make a distinction between Automating tactically and Automating Strategically. Because you can automate tactically without learning how to code. But if you want to automate strategically, and maximise your chances of success and have maximum flexibility then you will only get that when you learn how to code, and learn how to code well.

A lot of the "Test Automation" failure stories we hear are a result of attempting to automate strategically, but doing it badly. But we can learn to do it well.

I'm going to show you examples of using tools, and using tools in different ways. Because I want you to understand that many of the skills we develop when we look for tools, we also use when we code. And realising that is step one in moving from automating tactically to automating strategically.

You're going to see a worked example of moving from a tactically automated solution to one which is more strategic so you see the steps and the thought processes involved.

Title: "If you want to automate, you learn to code"

Blurb:

If you want to automate, you learn to code, and you learn to code well.

“Automate” doesn’t mean “Automate Testing” it means “Automate part of your test process”.

You need to learn to code to do that with the most options open to you.

We’ll look at some ‘we do this a lot’ and ‘we want to automate’ activities which we can use tools for. But we’ll also see that we are limited by the tools.

When we code, we can do a lot with minimum code, and gain a lot more flexibility.

Then we’ll cover how to think about learning to code.

- solve a problem quickly (automate tactically)
- solve a problem for the long term (automate strategically)

To work strategically we need to learn:

- to code well,
- understand refactoring,
- libraries vs frameworks,
- abstractions,
- etc.

This talk isn’t just for beginners, we’ll cover stuff that should make it useful for the experts in the audience.

We’ll cover a lot in 45 mins, with code examples and tool examples, and I’ll make it all pretty practical.

Other Notes

I want to go beyond the basics presented in “Java for Testers” and show ‘why’ and ‘how it helps’ to have coding skills.

Weave in pandocifier as an example throughout so the walkthrough at the end isn’t a surprise

Alan Richardson

- www.javafortesters.com
- www.seleniumsimplified.com
- www.eviltester.com

- www.compendiumdev.co.uk
- uk.linkedin.com/in/eviltester
- [@eviltester](http://twitter.com/eviltester)(<http://twitter.com/eviltester>)

My name is Alan Richardson and I'm a Test Consultant.

You can find me online in a bunch of places, these are the main URLs.

I help people test better. And part of that requires looking at how we can automate the stuff they do, and automate it better.

Yada Yada Yada. I've written some books. Online Training. Blah Blah Blah.

Look me up if you're interested in what I have to say.

“Automate” doesn’t mean “Automate Testing”

- dictionary.com/browse/automate

“to install automatic procedures, as for manufacturing or servicing;”

“Automate” doesn’t mean “Automate Testing”.

Without a specific object to contextualise the word ‘automate’, it means ‘install automatic procedures’.

When we test, we do stuff.

Some of that stuff we can do automatically.

“Automate” Make something happen automatically.

In the context of ‘testing’ it means “Automate part of your test process”.

You can do that strategically - take a long term view and plan for it, budget for it, etc.

And you can do that tactically - I have a problem, I want to solve it now, automate with quickly knocked up code or a tool.

But Automate does not mean “Automate Testing” it means automate some of the processes you use, some of the tasks that you do, when you test"

You don't *need* to 'code' to 'automate'

We can Automate by adding tools

You don't *need* to 'code' to 'automate'.

We can automate by adding tools.

The previous definition said "Install Automatic Procedures" that sounds a lot like 'install a tool to do something automatically'.

Someone else can 'code' a tool. And we can use that tool to 'automate' tasks or processes that overlap with the features of that tool.

I use Tools

- **FTP**
- **Text Editors**
- **etc.**
 - FTP

I normally use a GUI tool for copying files from my local machine to a remote machine using the FTP protocol.

I rarely use the command line `ftp` tool. But I have used it. And I've wrapped it in command line scripts to do automate parts of my workflow and process.

But I've never tried to implement the `ftp` protocol manually. In theory that might have been possible a long time ago with a really slow modem. The `ftp` protocol isn't really suited for manual implementation. We could use semaphore with flags, or morse code with lights or telegraph, but not `ftp`.

`ftp` is a protocol that demands an automated implementation. And I use tools to automate that part of my testing procedures.

- Text Editors

I have never manually written bytes to a disk to edit a file.

I have in the past used HEX editors, which is one step below a text editor, but even that is a tool.

I've used `Edlin` and `ed` a long time ago. Not pleasant. Line editors. But still a tool.

I've never tried to edit a file without some sort of automatic procedure installed.

We take 'tools' for granted

Because we use them so often

We take tools for granted because we use them so often. And its easy to forget that for pretty much any task, a tool will exist for it. And where we find one tool, we can probably find multiple tools.

Tools will offer different Graphical User Interfaces, slightly different functions, different capabilities, possibly APIs, possibly scripting capabilities.

So its worth investigating:

- what tools are you using?
- capabilities of your current tools
- alternatives to your current tools

So its worth investigating:

- what tools are you using?

Our operating system is a tool. Our Browsers are tools.

- capabilities of your current tools

They might do more than you think. There will be features you are not using.

- alternatives to your current tools.

You might increase your options. You might find better tools. Don't stick with the text editor that your operating system defaults to. Find one with features that support your wider workflow.

There are some processes we have to automate

And someone else might have automated them already.

That's a tool.

But tooling is a side effect.

What we are really trying to do, is do something.

And sometimes we want to do less, when we do something, so we want to automate parts of the process.

And sometimes we want nothing to do with the process so we want to automate as much of the process as possible.

And the more that we trust the tooling, the more we'll generally give the tooling to do.

And if someone has automated the process before us, and packed their automated solution as a 'tool' then we can use it.

But tools come with limitations

When I use a tool. I can only do, what the tool let's me do.

I can only do, what someone else coded the tool to do.

If I encounter a situation that the tool can't handle, then I can't deal with it.

E.g. Show browser, show cookies, I want to edit the cookies but I can't. I can't edit them. The tool doesn't let me. I am limited by my tool. But if I can extend the tool, with plugins then I might be OK. If I can't extend the tool. I'm stuck.

Requisite Variety

"Only variety can destroy variety" - Ross Ashby

[en.wikipedia.org/wiki/Variety_\(cybernetics\)](https://en.wikipedia.org/wiki/Variety_(cybernetics))

“Only variety can absorb variety” - Stafford Beer

To maximise our ‘requisite variety’, our ability to handle new and unexpected situations, with tooling we can:

- use multiple tools
 - use tools that support extension through scripts or plugins
 - find multiple ways to use the tools e.g. Unix tool chaining
 - learn to code and write our own tools
-

Strategy #1 - Build a toolbox with Multiple Tools

I used to do this.

<http://www.compendiumdev.co.uk/toolsinaction/Alternative%20Testing%20Tools%20List.pdf>

- build a list for myself
- build a company wiki page listing tools
- have a USB stick with portable tools on it
- subscribe to project announcements on sourceforge etc.

But I spent more time:

- looking for tools
- evaluating new tools

Than using the tools.

It felt like ‘busy work’.

So I eventually abandoned this practice so I could get some work done.

If I do want to do this I pretty much just download <http://portableapps.com/>

There is a difference between ‘Strategic’ and ‘Tactical’

Automate Tactically

solve a problem quickly

Automate Strategically

solve a problem for the long term

No definitions... but:

- Strategic - longer term
- Tactical - short term based on current environment

We can start thinking about automating in terms of ‘Strategic’ and ‘tactical’.

Toolbox building is ‘Strategic’

A long term strategy to identify tools.

And I have to commit to this.

This is a long term strategy to identify tools.

- Identify tools I ‘might’ need in the future
- Identify a range of tools for the same function to support variation
- Identify tools which are ‘easy to use’ and ‘quick fix’ and those which are ‘flexible’

And I have to commit to this long term.

If I stop, and let my tools list go out of date.

Then, I have nothing.

- My tool is out of date:
 - I haven’t used the tool for a while, it no longer works on this operating system.
 - My tool doesn’t use the most up to date protocol.
 - My tool doesn’t have the range of functions I need.

- My terrain scope is out of date:
 - I haven't kept an eye on technology progress
 - I don't have any tools to handle that technology

My FTP tool doesn't do FTPS!!!

If I'm lucky I will have the variety of tools I need to respond to the variability in the environment, but I need to constantly monitor my tool set to keep it current.

Strategies change and evolve

Tool box was a strategy I used to use, but don't any more. We don't have to stick to one strategy. Having multiple strategies, continuing to learn, try new approaches, have multiple long term strategies (learn Java, learn REST services, learn JavaScript, learn multiple things), pursue the learning in parallel.

Strategic Vs Tactical

We might want to be Agile, ninja-esque, highly tactical, handle a huge amount of variety in our stride.

Finding Tools can be a Tactical pursuit

If I learn how to find tools.

I can find tools when I need them.

Instead of building a toolbox.

I could build the skills of finding the tools I need, when I need them.

That sounds simple, but it leads to massively improved 'google-fu'

- What are you trying to do?

- What technology are you working on?
- What Operating System?
- What interpreters do you have installed? Go? Java? Python?

Use ‘Google’ or ‘Bing’ as your tool list.

Finding Tools when you need them

Is also a key step in learning how to automate.

Learning to code is not the first step in learning to automate.

If you don’t know what you are trying to achieve, then learning to code does not help you automate.

Learning to code is a strategic decision that will eventually lead to an improved ability to automate.

Finding tools when you need them is an easier skill to learn.

How do we find tools?

- What are you trying to achieve?
 - learn the domain language for the task
- Narrow by technology?
 - technology used
 - technology tool implemented in

We have to learn to speak the language of the tool builder if we want to find and use their tools.

Non-domain specific task

Search: “I want to copy a document to another machine”

learn: **server, transfer, file**

Worked example typing these queries into a search engine and following it up

Domain language for task

Search: “Tool to transfer file to server”

learn: **ftp, scp, sftp, ssh protocol, login, user account**

Specific Technology

“What does my specific server use?”

“What do I use?”

Search: “Windows Software to transfer file via scp ssh key”

Building these queries and chaining them is a skill

- the more we learn the better we get
- the better we understand the technology, the faster we find what we need
- technical staff take this skill for granted
- we also use this skill to find - IDE, Libraries, Build Tools, etc.

If you learn to program, without having this skill, your options of what you use, learn, apply your skills to are limited by the ‘tutor’.

Tactically finding tools is a skill that underpins the building of an ecosystem and tool suite for programming - you can build it separately and use it to help you automate even without programming.

Extend tools

Assuming that you can find tools

Favour tools that support extension

Learn the Extension possibilities for your chosen tools

Prefer tools that support extension through scripts or plugins.

Examples:

- OWasp ZAP Script
- FreeMind - Groovy Scripts
- Fiddler
- Browser - Whu? JavaScript Console e.g. TodoMVC example

APIs:

- MindMeister
- Jira

You pretty much need to have the ability to program to take advantage of the APIs, but if there is an API you want to use then it might give you the motivation and reason to learn to code.

Scripting a tool is an easy way to learn to code

We don't need to understand all the nuances of structuring a 'program' we can use the scaffolding provided by the tool.

I have used this approach to automate with FreePlane (Groovy), My own tools, Leo editor, sublime editor and more...

e.g. JavaScript in the browser

I don't know npm or node or gulp or angular or yams. JavaScript is not my programming strong point. But I know enough JavaScript to be dangerous.

Dangerous enough to automate applications from the browser dev tools.

- Example of todo mvc automating from browser console.

<http://todomvc.com/examples/backbone/>

- delete all the items

```
for (x = 0; x < app.todos.models.length; x++) {
  app.todos.models[x].destroy()
}
```

- Create 100 todos with a for loop

```
for (x = 0; x < 100; x++) {
  app.todos.create({
    title: "todo " + x,
    order: app.todos.nextOrder(),
    completed: false
  })
}
```

- Example of OWasp Proxy script

[read associated blog post](#)

```
/* Remove the X-Frame-Options header from response */
/*
```

Based on info from:

```
* https://github.com/zaproxy/community-scripts/tree/master/proxy
* https://groups.google.com/forum/#!topic/zaproxy-develop/_RZtqweBSws
```

```
*/
```

```
function proxyRequest(msg) {
  return true;
}
```

```
function proxyResponse(msg) {
  msg.getRequestHeader().setHeader("X-Frame-Options", null);
  return true;
}
```

- Example of Fiddler scripting
 - scroll through and see some examples

Tool Chaining

Don't under estimate the power of the command line

On Windows and Linux

Multiple ways to use the tools e.g. Unix tool chaining

- Bash scripts
- Dos Scripts

Call tools from programming languages.

We can usually hack something out by googling and copy and pasting.

```
dir *.bat /s
```

```
D:\Users\Alan\junit_java_Code\junit\InitialSeleniumTests\resources\deploy
```

```
D:\Users\Alan\sigist\2016\transcript\generate_output.pdf
```

e.g. here is one I use for working with pandoc

I created a batch file that we could drag and drop images on and it would call imagemagic to resize the image for testing.

And now for the meat of of it

“Give a man a fish you feed him for a day.

Teach a man to fish then you feed him for a lifetime.”

We all know the phrase if you give a man a fish you feed him for a day, if you teach a man to fish you feed him for a lifetime.

Well “Give a man a fish” - equates to the use of tools.

Teach a man to fish equates to - learning how to code and use libraries to solve problems.

The Ultimate Aim

“Give humankind the power to summon forth fish from the empty air”

We can do better than that.

We want to “give humankind the power to summon forth fish from the empty air”.

Then who knows what else they can do.

And that equates to learning to code really well and creating your own libraries.
(Learn to code well)

So that’s what we’re aiming for. But we’ll start with learning to code and use libraries.

Learn to code and write our own

This is a step up from using tools.

Basically the programming libraries are tools.

We learn to wrap them in our own code and use them in whatever way we can code.

I still need the ability to find the libraries, and find the most appropriate language to code in.

I still need the ability to work out what problem I’m trying to solve.

So by learning the tactical skill of finding tools then I can build on that to support my ‘learning to code’.

When people learn to code they focus on

programming language

IDEs

build process

‘making programs’

We don’t always ‘make programs’ when we automate

Most people who learn to code, do so because they are programmers.

They write programs.

Stand alone systems that are deployed.

We don’t necessarily need to do that when we automate.

We might. And that might be why people favour scripting languages - not because they are easier to learn but because they are easier to ‘run’ without the overhead of compiling, linking and converting into a .exe.

Pick your programming language for the right reasons

‘Easy to learn’?

‘Easy to run code’?

‘Easy to cobble together stuff that runs from copy and pasted examples off the internet’?

When we pick a language we need to make sure we pick it for the right reasons.

I've learned a lot of programming languages and the scripting languages weren't easier to 'learn' - not when I start doing 'advanced stuff' because pretty much all programming languages can do the same things and they all have advanced techniques and constructs.

Scripting languages are pretty easy to run code - easier than Java or .Net - but we have techniques for bypassing that.

Where Scripting languages really win is for Easy to cobble together stuff that runs from copy and pasted examples off the internet. Simple functionality that I could write by copy and pasting code examples from the internet, pretty much any language would do:

- batch files
- bash files
- autoit scripts
- perl

For me, Ruby and Python not so much because of the install process so I picked languages that were easy to install, or I didn't need to install. If you're on Mac then you probably have Ruby and Python installed - no so on Windows.

But hopefully we very quickly go beyond - cobbled together stuff and want to write good code.

I like Java because of all the support that I get when I code:

- IDE
- Code Completion
- Great Debugging
- Decompiling other libraries
- View source of libraries I'm using
- Evaluate running code

Stuff I don't get, when I work with many scripting languages - without going through hoops.

How to make Java Easy to run

Write an @Test method that does stuff

Run the @Test method

Tadah!

I'll quickly show you how to make Java easy to run.

I write an @Test method, which some code that does stuff.

I right click, and I run the test.

Oooh. Easy.

You can do this in pretty much every language. They are all easy to 'run' if you combine 'tools' like an IDE and a Test Harness.

You could do this in .Net as well, and in Ruby and Python etc. And you'll learn to code in a TDD style, that you won't if you are copy and paste scripting.

This is the entire approach I use in my book Java For Testers - we don't write programs, we write code that we run as @Test methods - and that code could be using libraries which help us automate processes.

Show this working as an Example in the presentation

For future reference see these blog posts:

- [How to learn to code Java without using a 'main' method](#)
- [An example of creating a 'tool' using @Test methods without building a Java application](<http://blog.javafortesters.com/2016/04/an-example-of-creating-tool-using-test.html>)
- [Is JUnit only for unit testing? What about system testing or UAT?](#)

So how do we pick a language?

- what is your system written in?
- what languages do people that you can ask for help know how to code?
- what examples have you found that do what you want to do?
- what tutorials or books have you found that you like?

If you don't have a lot of resources around you, then you might have to do some research.

But remember.

You don't have to stick to one language.

Particularly if you are 'getting things done'.

If you stay at the 'cobble stuff together from examples on the internet' level then it doesn't really matter which language you learn.

Automating Tactically Vs Automating Strategically

So let's quickly cover automating tactically vs automating strategically.

“Test Automation” - Automating Strategically

- long term
- paths through system

Very often when we discuss automating, we talk in terms of “Test Automation”.

And we're thinking in terms of Automating Strategically.

“As part of our test strategy we will automate the execution of paths which have been demonstrated as functionally acceptable in the past and by continuing to automate those paths with valid data we will check that the execution of those paths remains functionally acceptable.”

That takes a long term view of automating a system, for the purpose of checking it if changes, which is what most people mean by “Test Automation”.

And to do this we need to learn how to code, and code well.

The better we can code, the better we will create code that we can maintain long term and continue to keep it working as our system changes.

Good coding will lead to better long term automating.

And this is what most people think of when they think - I need to automate so I need to learn how to code.

Automate Tactically

- Automate to save time
- Automate to make your process more repeatable
- Automate to help you, or someone else

If you are just starting out, then learning how to think tactically about automating will probably serve you better.

Write code that saves time:

- starts up a browser and opens the app for you, logged in
- archives results and copies them to another disk
- generates reports from test notes
- stop all the processes, tidyup some files, restart the processes
- load in test data
- etc.

Whatever.

Anything that you do, that would make your life eaiser, if you automated it.

Tactical Solutions Might Become Strategic

- Maintainable
- Abstraction Layers
- Tested

Strategic Solutions can be used Tactically

- Adhoc exploration using abstraction layers
 - Temporary Data Exploration
-

Automate Tactically With

- Tools
- Chaining of Tools
- Scripting languages using tools and libraries
- `@Test` method using libraries

The better you know how to code, the more tools you know to call upon, the more libraries you know how to use, the more languages you have at your disposal.

The greater your requisite variety.

The more likely you can automate something tactically.

Warning - Tactical Solutions might not translate to Strategic Solutions without work

- the tactics we choose might not scale
- unreliable interaction
- time based synchronisation
- hacky code

Tactically I might create a quick knock up script using a really bad library that automates windows operating system sending keypresses and hard code waiting for a few seconds between actions.

Because only I will use that solution, it is a tactic for me, and if it fails I can recover.

That would suck as a strategic solution.

But someone somewhere will look at your tactical solution and say we should make this our strategy, because they don't know the difference and they don't know how to automate.

Don't be that person.

Danger - Opportunity Cost

- Some people view "Testing" as an opportunity cost of increased strategic "Automating"

- We could also consider “Not improving your ability to automate” an Opportunity cost of not automating strategically

We can refactor tactical solutions into strategic solutions

if we chose appropriate technology

It is possible to refactor tactical solutions into strategic solutions.

Generally, if we have the ability to create strategic solutions in the first place.

But - evaluate your tactical solution and make sure:

- it can synchronise effectively
- it has the capability to drive the application at the right level
- the programming language is good enough
- it is extendable
- etc. etc.

Examples of moving from tactical to strategic

Can't 'show you' many because:

- a lot of this is commercially sensitive
- very domain and project specific

I can't show you all the examples I've done this with.

I have tactical solutions for:

- working with mindmaps on mindmeister and freemind
 - downloading mindmaps from mindmeister
 - converting to freemind
 - uploading freemind mind maps to mindmeister
- online course platform interaction

- downloading the meta data on their platforms
 - mixing their APIs with automating the GUI to build the range of functionality I need
 - automating under their GUI with HTTP requests because they don't have APIs
 - streamlining my writing workflow
 - converting consultancy reports into different formats
 - splitting them up to have reports for different managers
 - tearing down and setting up environments
 - ... so much of the code I write is to automate tactically generally for myself and my team
-

How do I tend to build tactical solutions?

I can tell you that:

- I tend to use Java, not a scripting language
- Because... libraries, maven, JUnit
- ... I know it well

I tend to use Java because:

- I'm familiar with it
- I can build it into .jar easily if I need to
- I can easily add libraries with maven
- I can write 'code' without a GUI using the `@Test` approach
- Dependency management with maven means I can check in minimal amounts of code to the version control repository

We don't need scripting languages to do this.

And now an Example

Pandocifier

Automate Tactically

- batch shell scripts to automate pandoc
- pandocifier to run leanpub through pandoc

Refactor to Strategically Automated

- review, plan, refactor
 - keep it working as a tactical automated solution
-

Review Pandocifier

- Refactor to domain objects
 - Execute Pandoc
 - Inject Configuration
 - Externalise Configuration
 - External **Pandocifier** Object
 - Build into app
-

Refactor to domain objects

- There is one domain object - **BookTxtFile**
 - If I refactor to domain objects
 - fewer comments in the code
 - fewer lines of code in tactical solution
 - e.g. **PreviewMdFile**, **BookDetailsContentReader**
 - write Unit tests for the domain objects
-

Execute Pandoc

Instead of writing out instructions to the user to execute ‘tactic’.

```
// output the command to generate the book to console
System.out.println(
    "pandoc leanpubpreview.md -f markdown -s " +
    "-o leanpubpreview.pdf --toc");
```

The code could execute them:

```
Process p = Runtime.getRuntime().exec
    ("pandoc leanpubpreview.md -f markdown -s " +
    "-o leanpubpreview.pdf --toc")
```

Inject Configuration

Instead of hardcoding paths and file names (possibly pandoc arguments)

I could create a `PandocifierConfig` object with `getInputFilePath()` and `getOutputFileName()`

Externalise Configuration

Mechanism to create the `PandocifierConfig` from a file

- Properties file - easy to read with standard Java
 - text file - write a parser
 - JSON file - need an external library
-

External Pandocifier Object

- could create `Pandocifier` from `@Test` injecting config
 - `new Pandocifier(config).createPDF()`
-

Build into app

- create a main method with params that setup config
 - use `mvn package` to create a `.jar` file
 - use `java -jar pandocifier.jar` to execute
 - add to path and run from anywhere
-

Prioritise the ‘ideas’ to maximise tactical advantage

- Refactor, Execute, ...
- Inject Config, Execute Pandoc, ...
- Externalise Configuration, Execute Pandoc ...

Refactor, Execute, ...

Might spend a long time refactoring with no additional benefit

Inject Config, Execute Pandoc, ...

Cut down on time with easier config and creating the document.

Externalise Configuration, Execute Pandoc, ...

Have something I can execute from command line and wrap with other tools to increase my ‘tactical’ approach

Aims

My Aim with this is to:

- move towards something more ‘strategic’
- never break the tactical solution
- keep the tactical solution up to date with any benefits of moving strategic
e.g. extra features that arise as I move
- do it ‘fast’
- small chunks

- prioritise based on ‘value’ to me, as well as the project
-

Decide Prioritisation

- Inject Config,
- Execute Pandoc,
- Pandocifier,
- External Config

find the [code to support this decision on github](#)

Walkthrough of Changes for Config

- Create object to model config - store, not transform
- I Know that ‘if’ I create a `main` later, I can still use this
- Move the ‘hard coded’ stuff into an object
- Add it to an `@Before` to ‘simulate’ injection
- Possible future need for a PanDoc configuration

see [LeanPubConfigTest.java](#)

- Config create an object to model the configuration domain
- Stores the data, doesn’t transform it into ‘Pandocifier’ because it models the physical domain, not the usage domain
- Know that ‘if’ I create a `main` or an external configuration that I can still use this. We are often told YAGNI, but when refactoring, future use is important to consider to make sure we are heading in the right direction.
- Build it up by moving the ‘hard coded’ stuff into an object
- Add it to an `@Before` to ‘simulate’ injection
- Can see a possible need for a PanDoc configuration to configure command line, pdf naming etc. But I don’t NIY so I don’t create it yet because - YAGNI

see [LeanPubConfigTest.java](#)

Walkthrough of Changes for Execute

- Initially `Runtime.getRuntime().exec("...")`.
- Execution errors
- Rather than debug it I moved to `ProcessBuilder`
- New config for the `setPandocPath`
- Added opened output folder - new automated time saver

see [LeanPubPandocConfigExecuteTest.java](#)

- Initially tried creating a command line with `Runtime.getRuntime().exec("...")`.
- Execution kept failing when I tried to control path of the directory and the executable
- Rather than debug it I moved to `ProcessBuilder`
 - set arguments as separate params
 - set execution directory as `.directory(...)`
 - redirect output to current process
- Also meant adding a new config for the `setPandocPath`
- Also opened output folder after execution - new automated time saver

see [LeanPubPandocConfigExecuteTest.java](#)

Walkthrough of Changes for Pandocifier Object

- I create an internal class called `Pandocifier`
- construct it in the `@Before` with the config
- amend the `@Test` to use a `Pandocifier createPDF`
- automatically create the `createPDF`
- copy paste the `@Test` code into `createPDF`

see [LeanPubPandocConfigExecuteObjectTest.java](#)

Walkthrough of Changes for External Config

- Use new `@Ignore @Test` to create a property file
- Create a property file reader that returns a config as an internal class

- Now the `createPreviewMVP @Test` method looks like a `main` method

see [LeanPubPandocConfigExecuteObjectPropertiesTest.java](#)

- I use the existing hard coded strings to create a property file so I know it is in the right format - I do this in an `@Ignore` test so it will only run manually from the IDE when I want it to
- I create a property file reader that returns a config as an internal class
- normally I wouldn't create so many internal classes but because each class is a 'standalone' example I'm doing that.
- Now the `createPreviewMVP @Test` method looks like a `main` method. All I have to do is add the parameter processing

see [LeanPubPandocConfigExecuteObjectPropertiesTest.java](#)

Make it an App

- create a main method, call from `@Ignore MVP @Test`
- move the inner classes out to `main` packages
- change the pom.xml to have a build section for the manifest
- delete previous 'refactoring tests'
- `mvn package`
- because it is being [released on github](#) removed the 'leanpub' from .jar so people don't think it is official

Very often with main methods we don't 'test' them.

Temptation is to copy and paste code from `@Test` into `main`.

- create a main method by cut and paste code
 - move the inner classes out to `main` packages
 - rewrite the mvp 'test' call the main method - run main method here and test it interactively
 - change the pom.xml to have a build section for the manifest
 - `@Ignore` the MVP test so it doesn't interfere with the build
 - add some guards in the `main` args
 - delete previous 'refactoring tests'
 - `mvn package`
 - because it is being [released on github](#) removed the 'leanpub' from .jar so people don't think it is official
-

Retrospective

- always kept the tactical mvp class running and available
 - refactored to strategic solution
 - inner classes until used by multiple classes e.g. `main` and `MVP`
 - final strategic solution ‘wrapped’ as a ‘tactical’ `@Ignored` test
 - didn’t add many new features, didn’t ‘test’, still a lot to do if ‘other’ people want to use it
 - but now I can add it into CI for my local builds
-

Final Thoughts

- Automate tactically vs automate strategically
 - Use of Tools
 - Learn to program by extending tools
 - How to choose a programming language
 - How to move from tactical to strategic
-

Learn to code well

- still lots to learn, but its fun
- Abstractions
- Domain Driven Design
- Test Driven Development
- Design Patterns

Whatever you do. Make it add value.

Learning to code well, doesn’t just mean - learn a programming language and learn it properly (strategic) it also means learn how to program (just enough) that you can be effective in many languages (tactical).

- Automate tactically vs automate strategically
 - Automating a the most appropriate level of the application.
 - Automating with an appropriate library
 - Refactoring to appropriate levels of abstractions
-

Alan Richardson

- www.javafortesters.com
- www.seleniumsimplified.com
- www.eviltester.com
- www.compendiumdev.co.uk
- uk.linkedin.com/in/eviltester
- [[@eviltester](https://twitter.com/eviltester)](<http://twitter.com/eviltester>)